53. Design Waze — Real-Time Navigation & Traffic Platform

0) Requirements & Sizing

Functional

- **Real-time navigation**: Turn-by-turn directions with live traffic-aware routing
- **Live traffic map**: Aggregate GPS speed data from millions of drivers into road-segment speeds
- **Route calculation**: Fastest route using real-time + historical + predicted traffic
- **Dynamic rerouting**: Detect slowdowns mid-trip and recalculate automatically
- **User-reported events**: Accidents, police, hazards, road closures, speed cameras
- **Event verification**: Crowd-validate reports (thumbs up / "not there")
- **ETA estimation**: Continuously updated ETA based on live conditions
- **Carpool / ride matching**: Match drivers with riders heading the same direction
- **Map editing**: Community-driven map corrections and road edits
- **Alerts**: Approach-based alerts (speed cameras, hazards ahead on route)
- **Offline fallback**: Cache map tiles and last-known routes for connectivity gaps

Non-functional

Metric	Target
Scale	**150M monthly active users**, 50+ countries
Concurrent drivers	**20-30M simultaneously** sending GPS probes
GPS probe rate	**5-15M probes/sec** (one per 2-5 seconds per active driver)
Route calculation latency	< **1 second** for initial route
Reroute latency	< **3 seconds** detect slowdown + suggest alternative
Traffic speed freshness	< **30 seconds** from probe to map update
Event report propagation	< **10 seconds** to appear for nearby drivers
ETA accuracy	**±2-3 minutes** for trips under 1 hour

Availability	**99.95%** for navigation, **99.9%** for reporting
Global coverage	Multi-region, country-level data isolation where required

Capacity

```
GPS Probes:
  20M active drivers × 1 probe/3 sec = ~7M probes/sec
  Each probe: ~150 bytes (user_id, lat, lng, speed, heading, timestamp, road_segment_id)
  Ingestion throughput: 7M × 150B = ~1 GB/sec raw probe data

Road Network:
  ~50M road segments globally
  Each segment: segment_id, geometry, speed_limit, road_class, current_speed, ...
  Hot data (active segments): ~5M segments × 200B = ~1 GB in-memory

Traffic State:
  5M active road segments updated every 30 sec
  ~170K segment-speed updates/sec

User Reports:
  ~500K-1M reports/day globally
  ~10-50 reports/sec average, bursty (rush hour 10×)

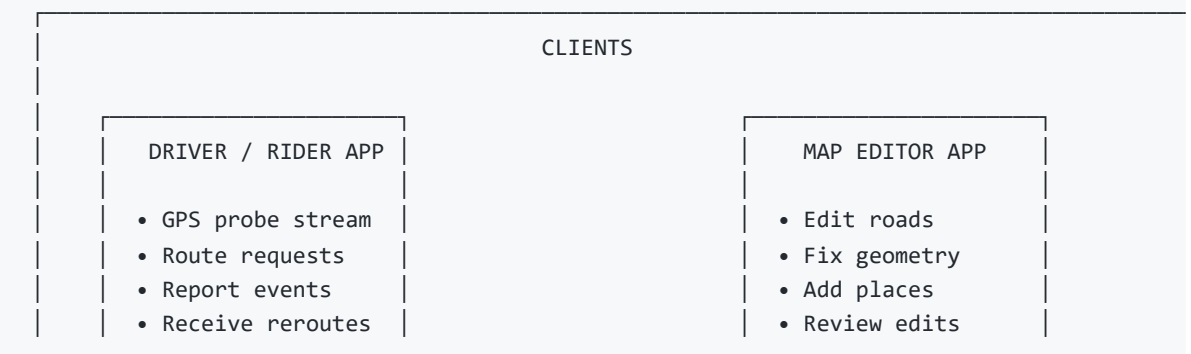
Map Tiles:
  ~100M tile requests/sec at peak (vector tiles, zoom levels 0-18)
  Pre-rendered + CDN cached: ~500TB tile storage

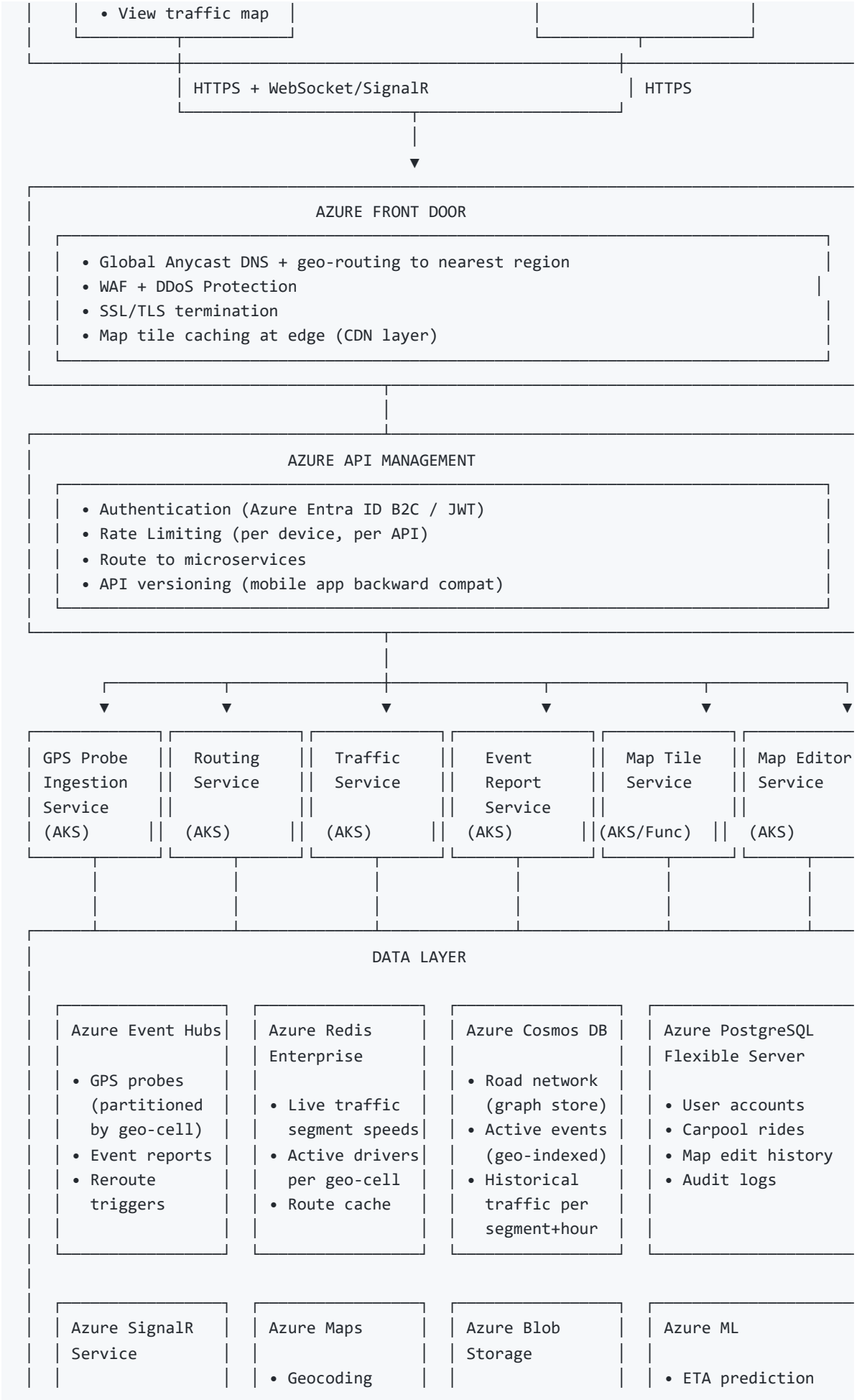
Storage:
  GPS history: 7M × 150B × 86,400 sec = ~90 TB/day raw → aggregated + TTL
  Traffic history: Keep 2 years for ML models → ~2 PB
  Road network: ~50 GB (graph + geometry)
  User reports: ~10 GB/day → 30 day retention for most
```

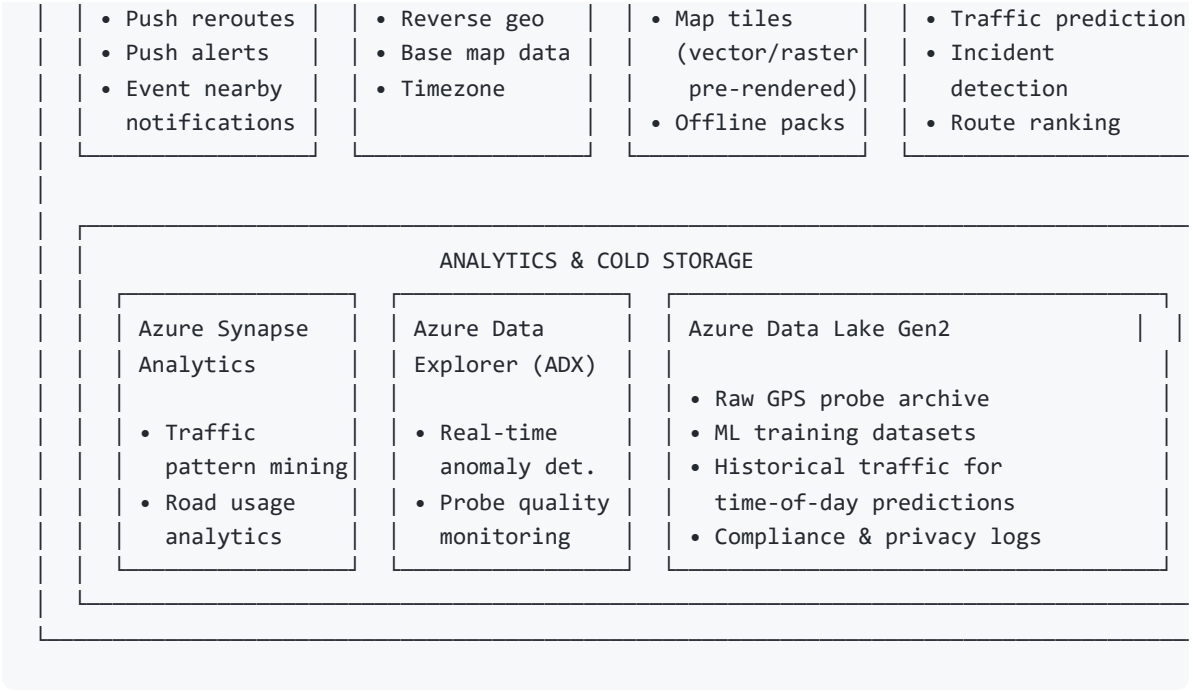
---

1) High-Level Architecture

Complete Azure Architecture




• View traffic map
HTTPS + WebSocket/SignalR
HTTPS
AZURE FRONT DOOR
• Global Anycast DNS + geo-routing to nearest region
• WAF + DDoS Protection
• SSL/TLS termination
• Map tile caching at edge (CDN layer)
AZURE API MANAGEMENT
• Authentication (Azure Entra ID B2C / JWT)
• Rate Limiting (per device, per API)
• Route to microservices
• API versioning (mobile app backward compat)
GPS Probe Ingestion Service (AKS)
Routing Service (AKS)
Traffic Service (AKS)
Event Report Service (AKS)
Map Tile Service (AKS/Func)
Map Editor Service (AKS)
DATA LAYER
Azure Event Hubs
• GPS probes (partitioned by geo-cell)
• Event reports
• Reroute triggers
Azure Redis Enterprise
• Live traffic segment speeds
• Active drivers per geo-cell
• Route cache
Azure Cosmos DB
• Road network (graph store)
• Active events (geo-indexed)
• Historical traffic per segment+hour
Azure PostgreSQL Flexible Server
• User accounts
• Carpool rides
• Map edit history
• Audit logs
Azure SignalR Service
Azure Maps
• Geocoding
Azure Blob Storage
Azure ML
• ETA prediction



Components Summary

Component	Azure Service	Purpose
Global Entry	**Azure Front Door**	Anycast DNS, WAF, DDoS, tile CDN
API Gateway	**Azure API Management**	Auth, rate limiting, routing
Compute	**Azure Kubernetes Service (AKS)**	All microservices
GPS Ingestion	**Azure Event Hubs**	7M probes/sec, partitioned by geo-cell
Live Traffic State	**Azure Redis Enterprise**	Segment speeds, driver index, route cache
Road Network & Events	**Azure Cosmos DB**	Graph data, geo-indexed events, traffic history
User & Edit Data	**Azure PostgreSQL**	Accounts, carpool, map edits
Real-time Push	**Azure SignalR Service**	Reroutes, alerts, event notifications
Map Tiles	**Azure Blob Storage + CDN**	Pre-rendered vector tiles, offline packs
Geocoding	**Azure Maps**	Address lookup, reverse geocoding
ML Models	**Azure ML**	ETA, traffic prediction, incident detection

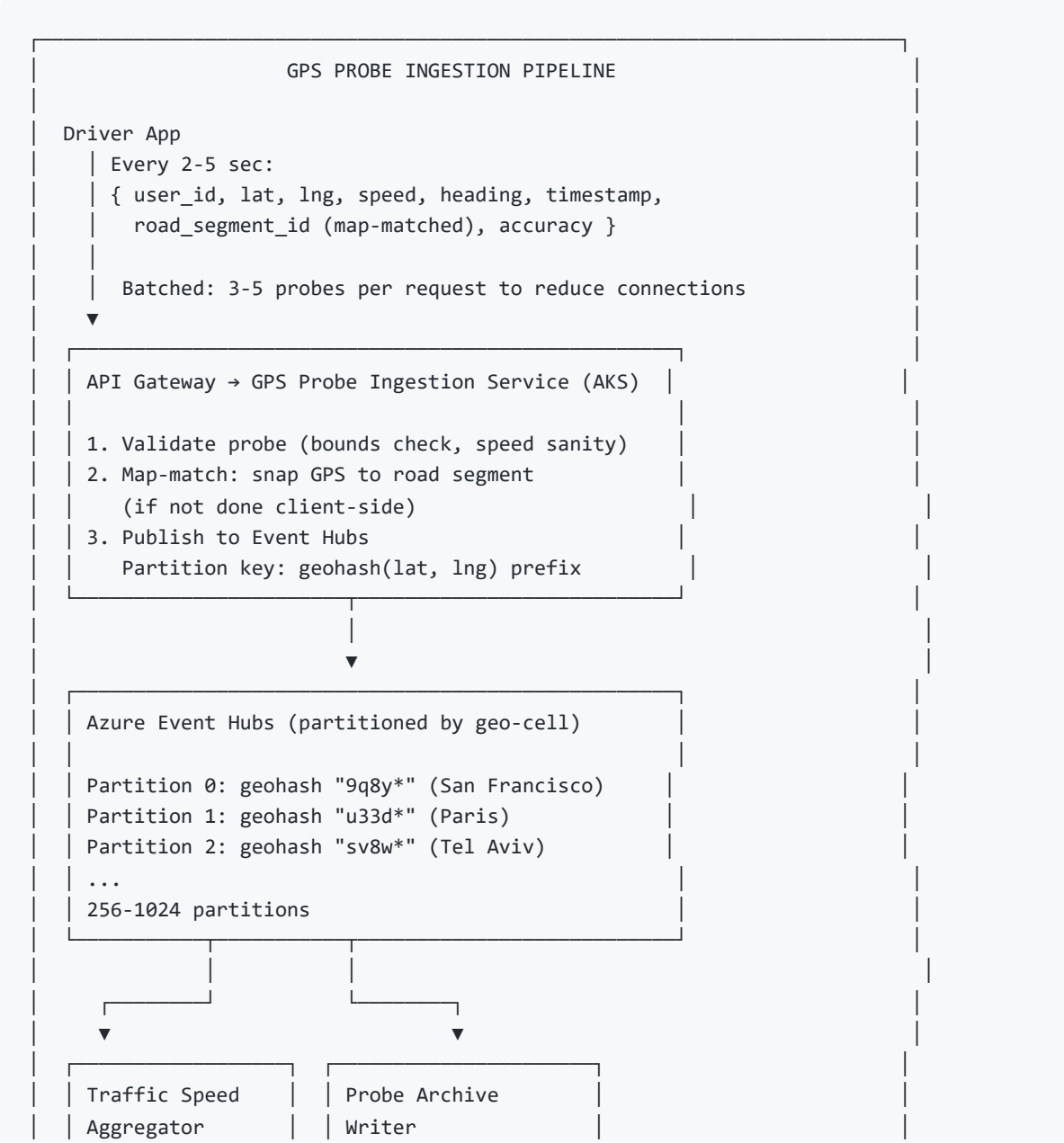
Stream Analytics	**Azure Data Explorer (ADX)**	Real-time probe quality, anomaly detection
Batch Analytics	**Azure Synapse**	Historical patterns, BI dashboards
Archive	**Azure Data Lake Gen2**	Raw probes, ML training data

---

2) Core Components Deep Dive

2.1 GPS Probe Ingestion Pipeline

The **heartbeat** of Waze — every active driver sends GPS probes every 2-5 seconds. This is the highest-throughput path in the entire system.



```
│   │ (AKS / Stream)  │  │ (AKS)                │                          │
│   │                  │  │                      │                           │
│   │ Sliding window:  │  │ Batch write to       │                           │
│   │ 30-sec tumbling  │  │ Data Lake Gen2       │                           │
│   │ aggregate per    │  │ (Parquet, partitioned │                           │
│   │ road segment     │  │  by date + geo-cell) │                           │
│   │                  │  │                      │                           │
│   │ Output:          │  │ For ML training +    │                           │
│   │ segment_id →     │  │ historical analysis  │                           │
│   │ avg_speed,       │  │                      │                           │
│   │ probe_count,     │  │                      │                           │
│   │ confidence       │  │                      │                           │
│   └────────┬─────────┘  └──────────────────────┘                           │
│            │                                                               │
│            ▼                                                               │
│   ┌──────────────────┐                                                     │
│   │ Redis Enterprise │                                                     │
│   │                  │                                                     │
│   │ Key: seg:{id}    │                                                     │
│   │ Val: {speed: 45, │                                                     │
│   │       count: 23, │                                                     │
│   │       updated_at}│                                                     │
│   │ TTL: 5 min       │                                                     │
│   │                  │                                                     │
│   │ If speed drops   │                                                     │
│   │ >40% vs prev →   │                                                     │
│   │ publish SLOWDOWN │                                                     │
│   │ event            │                                                     │
│   └──────────────────┘                                                     │
│                                                                            │
└────────────────────────────────────────────────────────────────────────────┘
```

Map Matching

```
Problem: GPS coordinates are noisy (±10m). A probe at (34.0522, -118.2437)
         could be on any of 3 nearby roads. Which road is the driver on?

Map Matching: Snap GPS points to the correct road segment.

  Raw GPS probes:          After map matching:
       .  . .                    ──────────── Road A
      .       .              .  .  .  .  .  .
     .    .    .                ──────────── Road B
      .       .
       .  . .

Algorithm: Hidden Markov Model (HMM)
  States:    candidate road segments near GPS point
  Emission:  probability based on distance from GPS to road
  Transition: probability based on route continuity

  Uses Viterbi algorithm to find most likely sequence of road segments.

  Client-side map matching:
```

```
    ✅ Reduces server load (no raw GPS to process)
    ✅ Can use phone's sensors (gyroscope, accelerometer) for better accuracy
    ❌ Requires up-to-date road network data on device
    ❌ Battery impact from local computation

  Server-side map matching:
    ✅ Always uses latest road network
    ✅ Can use multi-probe context (smoother results)
    ❌ Higher server load
    ❌ Requires sending raw GPS (more bandwidth + privacy concern)

  Waze approach: Hybrid — client does basic matching, server corrects.
```

2.2 Traffic Speed Computation

```
How Waze Knows Traffic Speed on Every Road:

  Road Network = directed graph of "segments"
  Each segment = one direction of a road between two junctions

  Example: "Main St between 1st Ave and 2nd Ave, eastbound"
    segment_id: seg_5829371
    length: 250 meters
    speed_limit: 50 km/h
    lanes: 2
    road_class: "primary"

  ┌──────────────────────────────────────────────────────────────┐
  │  TRAFFIC SPEED AGGREGATION (per segment, every 30 seconds)  │
  │                                                              │
  │  Input: all GPS probes map-matched to this segment           │
  │                                                              │
  │  Probes in last 30s for seg_5829371:                         │
  │    Driver A: speed = 42 km/h                                 │
  │    Driver B: speed = 38 km/h                                 │
  │    Driver C: speed = 40 km/h                                 │
  │    Driver D: speed = 5 km/h  ← outlier (probably parking)    │
  │    Driver E: speed = 41 km/h                                 │
  │                                                              │
  │  Aggregation:                                                │
  │    1. Remove outliers (z-score > 2 or < 2 km/h)              │
  │    2. Weighted median (recent probes weighted higher)        │
  │    3. Confidence = f(probe_count, time_spread)               │
  │                                                              │
  │  Result:                                                     │
  │    seg_5829371 → {                                           │
  │      speed: 40 km/h,                                         │
  │      free_flow_speed: 50 km/h,                               │
  │      congestion_level: 0.80 (speed / free_flow),             │
  │      probe_count: 4,                                         │
  │      confidence: 0.85,                                       │
  │      updated_at: 2024-01-15T08:30:00Z                        │
  │    }                                                         │
  │                                                              │
```

```
   │  Color mapping for map display:                                  │
   │    Green:  congestion > 0.75 (flowing freely)                    │
   │    Orange: congestion 0.40-0.75 (moderate traffic)               │
   │    Red:    congestion 0.15-0.40 (heavy traffic)                  │
   │    Dark Red: congestion < 0.15 (standstill)                      │
   │    Gray:   no recent probes (use historical/predicted)           │
   └──────────────────────────────────────────────────────────────────┘

When No Live Probes Are Available:
  Fall back to historical traffic model:
    speed = historical_avg(segment_id, day_of_week, time_of_day)

  Example: "seg_5829371 on Tuesday at 08:30" historically averages 35 km/h

  Historical model stored in Cosmos DB:
    Partition key: segment_id
    Document: { segment_id, day_of_week, hour_slot, avg_speed, stddev }
    → 50M segments × 7 days × 24 hours = 8.4B rows (pre-aggregated)
```

2.3 Routing Engine

```
┌──────────────────────────────────────────────────────────────────────┐
│                      ROUTING ENGINE                                  │
│                                                                      │
│  Input: origin (lat, lng) + destination (lat, lng)                   │
│  Output: ordered list of road segments + turn instructions + ETA     │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │ ROAD NETWORK GRAPH                                           │    │
│  │                                                              │    │
│  │ Nodes: ~300M intersections/junctions globally                │    │
│  │ Edges: ~500M directed road segments                          │    │
│  │ Edge weight: travel_time (NOT distance)                      │    │
│  │                                                              │    │
│  │ Weight = segment_length / current_speed                      │    │
│  │                                                              │    │
│  │ Live:       current_speed from Redis (if fresh probes exist) │    │
│  │ Historical: predicted speed from ML model (day+hour)         │    │
│  │ Static:     speed_limit × road_class_factor (last resort)    │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │ ALGORITHM: Contraction Hierarchies (CH) + A*                 │    │
│  │                                                              │    │
│  │ Why not plain Dijkstra?                                      │    │
│  │   Dijkstra on 500M edges → seconds per query ❌              │    │
│  │   CH precomputes "shortcuts" → milliseconds per query ✓      │    │
│  │                                                              │    │
│  │ Contraction Hierarchies:                                     │    │
│  │                                                              │    │
│  │ PRECOMPUTE (offline, periodic):                              │    │
│  │   1. Order nodes by "importance" (highways > local streets)  │    │
│  │   2. "Contract" less important nodes by adding shortcuts     │    │
│  │                                                              │    │
```

```
  │   │   Original:  A ──5──→ B ──3──→ C                                  │  │
  │   │   Contract B: A ─────────8─────────→ C  (shortcut, skip B)         │  │
  │   │                                                                    │  │
  │   │   Result: hierarchical graph where searching upward               │  │
  │   │   (to highways) + downward (to destination) is very fast           │  │
  │   │                                                                    │  │
  │   │ QUERY (online):                                                    │  │
  │   │   Bidirectional search: forward from origin, backward from         │  │
  │   │   destination, meet in the middle at a "highway" node.             │  │
  │   │   Typical: explore ~1000 nodes instead of millions.                │  │
  │   │   Latency: 1-10ms per route on modern hardware.                    │  │
  │   │                                                                    │  │
  │   │ TRAFFIC-AWARE CH (Customizable CH):                                │  │
  │   │   Standard CH uses fixed weights → can't change with traffic       │  │
  │   │   Customizable CH: precompute structure, swap in live weights      │  │
  │   │   → Update edge weights every 30-60 sec from Redis                 │  │
  │   │   → Re-customize the CH metric (fast: O(n) not full rebuild)       │  │
  │   └────────────────────────────────────────────────────────────────────┘  │
  │                                                                           │
  │   ┌────────────────────────────────────────────────────────────────────┐  │
  │   │ MULTI-ROUTE GENERATION                                             │  │
  │   │                                                                    │  │
  │   │ Waze shows 2-3 alternative routes. How?                            │  │
  │   │                                                                    │  │
  │   │ 1. Compute shortest path P1                                        │  │
  │   │ 2. Penalize edges in P1 (multiply weight × 1.5)                    │  │
  │   │ 3. Compute again → P2 (different path)                             │  │
  │   │ 4. Penalize P1 ∪ P2 edges → P3                                     │  │
  │   │ 5. Rank by: ETA, reliability, road quality                         │  │
  │   │                                                                    │  │
  │   │ Also considers:                                                    │  │
  │   │   • Toll roads (user preference: avoid/allow)                      │  │
  │   │   • Unpaved roads (avoid by default)                               │  │
  │   │   • Restricted turns / time-based restrictions                     │  │
  │   │   • User-reported road closures                                    │  │
  │   └────────────────────────────────────────────────────────────────────┘  │
  │                                                                           │
  └───────────────────────────────────────────────────────────────────────────┘
```

Route Caching

```
Not every route needs a full computation:

  Cache hierarchy:
  ┌──────────────────────────────────────────────────────┐
  │ L1: Client cache (on-device)                         │
  │     • Active route stored locally                    │
  │     • Recently searched routes (LRU, 50 entries)     │
  │     • TTL: duration of trip                          │
  ├──────────────────────────────────────────────────────┤
  │ L2: Redis route cache (server-side)                  │
  │     • Key: hash(origin_cell, dest_cell, time_bucket) │
  │     • Popular commuter routes pre-computed           │
  │     • TTL: 5-10 min (invalidated on traffic change)  │
```

```
│     • Hit rate: ~30-40% during rush hours                  │
├────────────────────────────────────────────────────────────┤
│ L3: Full route computation (CH algorithm)                  │
│     • Cache miss → compute fresh route                     │
│     • Store result in L2 for next user                     │
└────────────────────────────────────────────────────────────┘
```

2.4 Dynamic Rerouting

```
┌──────────────────────────────────────────────────────────────────────┐
│                    DYNAMIC REROUTING                                 │
│                                                                      │
│  The #1 feature that makes Waze valuable — automatic rerouting       │
│  when conditions change mid-trip.                                    │
│                                                                      │
│  TRIGGER DETECTION:                                                  │
│                                                                      │
│  1. Speed Drop Detection (from Traffic Speed Aggregator)             │
│     IF segment.speed drops > 40% vs 1-minute-ago                     │
│        AND segment is on active_route of user X                      │
│     → Publish reroute_trigger(user_id, segment_id) to Event Hubs     │
│                                                                      │
│  2. User-Reported Event on Route                                     │
│     IF new event (accident, closure) is on user X's planned route    │
│     → Publish reroute_trigger(user_id, event_id)                     │
│                                                                      │
│  3. Periodic Route Check (every 2-5 min)                             │
│     For each active navigation session:                              │
│       recalculate ETA with current traffic                           │
│       IF new_ETA > current_ETA + threshold (e.g., 5 min)             │
│       → Check for better alternative route                           │
│                                                                      │
│  REROUTE PIPELINE:                                                   │
│                                                                      │
│  reroute_trigger                                                     │
│    │                                                                 │
│    ▼                                                                 │
│  Reroute Service (AKS)                                               │
│    │                                                                 │
│    ├─ 1. Get user's current position + remaining route               │
│    ├─ 2. Compute alternative route from current position             │
│    ├─ 3. Compare: is alternative significantly better?               │
│    │     IF alternative_ETA < current_remaining_ETA - 2 min          │
│    │     → YES, suggest reroute                                      │
│    │                                                                 │
│    ├─ 4. Push via Azure SignalR:                                     │
│    │     { type: "reroute_suggestion",                               │
│    │       new_route: [...segments],                                 │
│    │       saved_time: "7 min",                                      │
│    │       reason: "accident on Route 101" }                         │
│    │                                                                 │
│    └─ 5. Client shows: "Faster route available, saves 7 min"         │
│         User taps → switch to new route                              │
│         (or auto-switch if preference enabled)                       │
```

```
┌──────────────────────────────────────────────────────────────────┐
│                                                                  │
│  PREVENTING REROUTE STORMS:                                      │
│                                                                  │
│  Problem: If 10,000 drivers get rerouted to the same side street,│
│           that street now becomes congested too.                 │
│                                                                  │
│  Solution: Load-balanced rerouting                               │
│    1. Track capacity per road segment (lanes × max_flow)         │
│    2. As reroutes are issued, increment projected_flow           │
│    3. When projected_flow > 80% capacity, stop suggesting        │
│       that alternative → force next users to a different route   │
│    4. Distribute traffic across 2-3 alternatives proportionally  │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

2.5 User-Reported Events

```
┌──────────────────────────────────────────────────────────────────┐
│                    USER EVENT REPORTING SYSTEM                   │
│                                                                  │
│  Event Types:                                                    │
│    🚨 Accident (major/minor)        🚧 Road closure              │
│    👮 Police                        ⚠️ Hazard (object on road)   │
│    📸 Speed camera (fixed/mobile)   🚗 Traffic jam               │
│    🏗️ Road construction             🌊 Flood                     │
│                                                                  │
│  REPORT FLOW:                                                    │
│                                                                  │
│  Driver taps "Report" → selects type → optional comment/photo    │
│    │                                                             │
│    ▼                                                             │
│  ┌──────────────────────────────────────────────────────┐        │
│  │ Event Report Service (AKS)                           │        │
│  │                                                      │        │
│  │ 1. Create event:                                     │        │
│  │    { event_id, type: "accident", lat, lng,           │        │
│  │      road_segment_id, direction, reporter_id,        │        │
│  │      timestamp, confidence: 0.5, ttl: 60min }        │        │
│  │                                                      │        │
│  │ 2. Store in Cosmos DB (geo-indexed)                  │        │
│  │    Partition key: geohash_prefix (4 chars)           │        │
│  │    Geo-spatial index on location                     │        │
│  │                                                      │        │
│  │ 3. Publish to Event Hubs → "event-reports" topic     │        │
│  │                                                      │        │
│  │ 4. Notify nearby drivers via SignalR:                │        │
│  │    Query: drivers within 5 km heading toward event   │        │
│  │    Push: { type: "alert", event_type: "accident",    │        │
│  │            distance_ahead: "2.3 km", segment_id }    │        │
│  └──────────────────────────────────────────────────────┘        │
│                                                                  │
│  CROWD VERIFICATION:                                             │
│                                                                  │
│  ┌──────────────────────────────────────────────────────┐        │
```

```
│   │ When a driver passes an event location:                          │      │
│   │   App asks: "Police reported ahead. Still there?"                 │      │
│   │   Options: 👍 (confirm) / 👎 (not there)                          │      │
│   │                                                                   │      │
│   │ Confidence scoring:                                               │      │
│   │                                                                   │      │
│   │   Initial report:          confidence = 0.5                       │      │
│   │   + each 👍 confirmation:  confidence += 0.15                     │      │
│   │   + each 👎 rejection:     confidence -= 0.20                     │      │
│   │   + GPS speed corroboration:                                      │      │
│   │     IF drivers near event are slow → confidence += 0.10           │      │
│   │     IF drivers near event are normal → confidence -= 0.10         │      │
│   │                                                                   │      │
│   │   IF confidence > 0.8 → show on map for all users                 │      │
│   │   IF confidence < 0.2 → remove from map                           │      │
│   │   IF no interaction for TTL → expire automatically                │      │
│   │                                                                   │      │
│   │ Reporter reputation:                                              │      │
│   │   Reporters with high accuracy history get higher                 │      │
│   │   initial confidence (0.6-0.7 vs 0.5 for new users)               │      │
│   └───────────────────────────────────────────────────────────────────┘      │
│                                                                              │
│  AUTO-DETECTION (no user report needed):                                     │
│                                                                              │
│   ┌───────────────────────────────────────────────────────────────────┐      │
│   │ ML-based incident detection from GPS probes:                      │      │
│   │                                                                   │      │
│   │ Pattern: Multiple drivers suddenly braking at same spot           │      │
│   │   → Likely accident or hazard                                     │      │
│   │                                                                   │      │
│   │ Pattern: All drivers suddenly stopping on highway                 │      │
│   │   → Likely road closure or major accident                         │      │
│   │                                                                   │      │
│   │ Pattern: Drivers detour around a road segment                     │      │
│   │   → Likely road closure or flooding                               │      │
│   │                                                                   │      │
│   │ These generate "system events" with confidence 0.4                │      │
│   │ (lower than user reports, boosted by confirmations)               │      │
│   └───────────────────────────────────────────────────────────────────┘      │
│                                                                              │
└──────────────────────────────────────────────────────────────────────────────┘
```

2.6 ETA Estimation

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          ETA ESTIMATION                                      │
│                                                                              │
│  ETA is recalculated every 30-60 seconds during active navigation.           │
│                                                                              │
│  ETA = Σ (segment_length / predicted_speed) for each segment ahead           │
│        + Σ (turn_penalty) for each turn/intersection                         │
│        + traffic_light_delay × num_traffic_lights                            │
│                                                                              │
│  Where predicted_speed for each segment =                                    │
```

```
│    IF (live_speed exists AND confidence > 0.5):                          │
│      predicted = live_speed                                              │
│    ELIF (ML_predicted_speed exists):                                     │
│      predicted = ML_model(segment, day_of_week, time, weather)           │
│    ELSE:                                                                 │
│      predicted = historical_avg(segment, day, hour)                      │
│                                                                          │
│  ML ETA Model (Azure ML):                                                │
│                                                                          │
│  Features:                                                               │
│    • Segment-level: road_class, lanes, speed_limit, length               │
│    • Temporal: day_of_week, hour, is_holiday, is_rush_hour               │
│    • Live: current_speed, congestion_level, events_ahead                 │
│    • Weather: rain, snow, visibility (from weather API)                  │
│    • Global: overall city congestion index                               │
│                                                                          │
│  Training data:                                                          │
│    Historical trips: actual_travel_time vs predicted_travel_time         │
│    Billions of data points from Data Lake Gen2                           │
│                                                                          │
│  Model: Gradient Boosted Trees (LightGBM) or Transformer                 │
│    Trained per-city (traffic patterns are city-specific)                 │
│    Retrained weekly, A/B tested before deployment                        │
│                                                                          │
│  Accuracy target: ±2-3 min for trips < 1 hour                            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

2.7 Map Tile Serving

```
┌──────────────────────────────────────────────────────────────────────────┐
│                      MAP TILE SERVING                                    │
│                                                                          │
│  The map display layer — rendering roads, buildings, labels.             │
│                                                                          │
│  VECTOR TILES (not raster):                                              │
│    Waze uses vector tiles — sends geometry + metadata, client            │
│    renders. Smaller than images, zoom-smooth, styleable.                 │
│                                                                          │
│  Tile pyramid:                                                           │
│    Zoom 0:  1 tile  (whole world)                                        │
│    Zoom 5:  1,024 tiles (country level)                                  │
│    Zoom 10: 1M tiles (city level)                                        │
│    Zoom 15: 1B tiles (street level)                                      │
│    Zoom 18: 69B tiles (building level) — NOT all pre-rendered            │
│                                                                          │
│  Serving Architecture:                                                   │
│                                                                          │
│    Client requests: GET /tiles/{z}/{x}/{y}.mvt                           │
│      │                                                                   │
│      ▼                                                                   │
│    Azure Front Door (CDN)                                                │
│      │ Cache HIT (95%+ of tile requests) → return immediately            │
│      │ Cache MISS ↓                                                      │
```

```
│      ▼                                                                  │
│    Tile Service (AKS / Azure Functions)                                 │
│      │                                                                  │
│      ├─ Pre-rendered tile exists in Blob Storage? → return              │
│      │                                                                  │
│      └─ Dynamic tile generation:                                        │
│         Query road geometry from Cosmos DB for this bbox                │
│         Encode as Mapbox Vector Tile (MVT) format                       │
│         Store in Blob Storage for next request                          │
│         Return to client                                                │
│                                                                          │
│  TRAFFIC OVERLAY (separate layer):                                      │
│    Traffic colors are NOT baked into map tiles.                          │
│    Served as a separate real-time layer:                                │
│                                                                          │
│    Client polls: GET /traffic/{z}/{x}/{y}.mvt?t={timestamp}            │
│    Response: segment geometries + congestion_level                       │
│    Refresh: every 30 seconds                                            │
│    CDN TTL: 15-30 seconds (short, traffic changes fast)                 │
│                                                                          │
│  EVENT OVERLAY (separate layer):                                        │
│    Real-time push via SignalR (not polling):                            │
│    Events within viewport → render icons on map                         │
│    Updates when events created/expired/moved                            │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

---

3) Data Models

Road Network (Cosmos DB — NoSQL API)

```
// Road Segment document
{
  "id": "seg_5829371",
  "partitionKey": "region_us_west",
  "type": "segment",
  "geometry": {
    "type": "LineString",
    "coordinates": [[-118.2437, 34.0522], [-118.2445, 34.0528], ...]
  },
  "properties": {
    "road_name": "Main Street",
    "direction": "eastbound",
    "road_class": "primary",        // motorway, primary, secondary, residential
    "speed_limit_kmh": 50,
    "lanes": 2,
    "length_m": 250,
    "toll": false,
    "unpaved": false,
    "restrictions": [
      { "type": "no_left_turn", "from_segment": "seg_5829370" },
```

```
      { "type": "time_restricted", "hours": "07:00-09:00", "days": "Mon-Fri" }
    ]
  },
  "connections": {
    "from_node": "node_192837",
    "to_node": "node_192838",
    "connected_segments": ["seg_5829372", "seg_5829375"]
  },
  "_ts": 1705312200
}
```

Live Traffic State (Redis Enterprise)

```
// Current speed per segment (updated every 30 sec)
SET seg:5829371:speed "{\"speed\":40,\"ff_speed\":50,\"congestion\":0.80,\"probes\":4,\"

// Active drivers in geo-cells (for nearby driver queries)
GEOADD drivers:active -118.2437 34.0522 "user_abc123"

// Active navigation sessions
HSET nav:user_abc123 "route" "[seg_1,seg_2,seg_3,...]"
HSET nav:user_abc123 "current_segment" "seg_2"
HSET nav:user_abc123 "eta" "2024-01-15T09:15:00Z"
HSET nav:user_abc123 "dest_lat" "34.0689"
HSET nav:user_abc123 "dest_lng" "-118.4452"
EXPIRE nav:user_abc123 7200  // 2 hour TTL
```

User-Reported Events (Cosmos DB — geo-indexed)

```
{
  "id": "evt_20240115_abc",
  "partitionKey": "9q8y",            // geohash prefix
  "type": "event",
  "event_type": "accident",
  "subtype": "major",
  "location": {
    "type": "Point",
    "coordinates": [-118.2437, 34.0522]
  },
  "road_segment_id": "seg_5829371",
  "direction": "eastbound",
  "reporter_id": "user_xyz789",
  "confidence": 0.75,
  "votes": { "thumbs_up": 5, "thumbs_down": 1 },
  "created_at": "2024-01-15T08:30:00Z",
  "expires_at": "2024-01-15T09:30:00Z",
  "description": "Two cars, right lane blocked",
  "photo_url": "https://blob.../evt_photo_abc.jpg",
  "_ts": 1705312200
}
```

Historical Traffic (Cosmos DB — pre-aggregated)

```
{
  "id": "seg_5829371_tue_0830",
  "partitionKey": "seg_5829371",
  "segment_id": "seg_5829371",
  "day_of_week": "tuesday",
  "hour_slot": "08:30",
  "avg_speed_kmh": 35.2,
  "p10_speed": 22.0,          // 10th percentile (bad day)
  "p50_speed": 35.0,          // median
  "p90_speed": 48.0,          // 90th percentile (good day)
  "sample_count": 15420,
  "last_updated": "2024-01-14"
}
```

---

4) Key Algorithms & Trade-offs

Contraction Hierarchies vs. A* vs. Dijkstra

```
┌──────────────────────┬──────────────┬──────────────────┬──────────────────────────┐
│ Algorithm            │ Query Time   │ Preprocessing    │ Dynamic Weights?         │
├──────────────────────┼──────────────┼──────────────────┼──────────────────────────┤
│ Dijkstra             │ O(E log V)   │ None             │ ✅ Yes                   │
│                      │ ~seconds     │                  │ (but too slow)           │
├──────────────────────┼──────────────┼──────────────────┼──────────────────────────┤
│ A*                   │ O(E log V)   │ None             │ ✅ Yes                   │
│                      │ ~100ms       │                  │ (still slow at scale)    │
├──────────────────────┼──────────────┼──────────────────┼──────────────────────────┤
│ Contraction          │ O(k log k)   │ Hours (full)     │ ⚠️ Customizable CH       │
│ Hierarchies (CH)     │ ~1-10ms      │ Seconds (update  │ allows weight updates    │
│                      │ k ≈ 1000     │ weights only)    │ without full rebuild     │
├──────────────────────┼──────────────┼──────────────────┼──────────────────────────┤
│ CH + Live Traffic    │ ~5-15ms      │ ~30-60 sec       │ ✅ Metric update         │
│ (Customizable)       │              │ periodic         │ every 30-60 sec          │
└──────────────────────┴──────────────┴──────────────────┴──────────────────────────┘

Waze uses Customizable Contraction Hierarchies:
  • Structure built once (hours) — node ordering and shortcuts
  • Metric (edge weights) updated every 30-60 sec from live traffic
  • Query: bidirectional search on hierarchy — milliseconds
```

Geo-Partitioning Strategy

```
How to partition 7M probes/sec across infrastructure:

  Strategy: Geohash-based partitioning
```

```
Geohash "9q8y" = ~5km × 5km cell in San Francisco
Geohash "9q8"  = ~40km × 20km (city-level)
Geohash "9q"   = ~600km × 600km (region-level)

Event Hubs partitioning:
  Partition key = geohash prefix (4-5 chars)
  → All probes for a city go to same partition set
  → Traffic aggregation is partition-local (no cross-partition joins)

Benefits:
  ✅ Geographic locality → probes for same road → same partition
  ✅ Routing queries are geographically bounded
  ✅ Scale by adding partitions for high-density regions
  ✅ Per-region deployment for data residency compliance

Trade-off:
  ❌ Hot spots (Manhattan vs. rural Kansas — uneven load)
  Fix: Subdivide hot cells into finer geohash (adaptive)
  ❌ Routes crossing partition boundaries need federation
  Fix: Routing engine reads from multiple region caches
```

Event TTL & Garbage Collection

```
Not all events last forever. TTL management:

┌──────────────────────┬──────────────┬──────────────────────────────────┐
│ Event Type           │ Default TTL  │ Extension Rules                  │
├──────────────────────┼──────────────┼──────────────────────────────────┤
│ Police               │ 30 min       │ +15 min per confirmation         │
│ Speed camera (fixed) │ Permanent    │ Removed only by map editors      │
│ Speed camera(mobile) │ 60 min       │ +30 min per confirmation         │
│ Accident (minor)     │ 45 min       │ +20 min per confirmation         │
│ Accident (major)     │ 120 min      │ +30 min, or until cleared        │
│ Hazard               │ 60 min       │ +15 min per confirmation         │
│ Road closure         │ 240 min      │ Extended by confirmations        │
│ Traffic jam          │ Auto         │ Removed when speed recovers      │
│ Construction         │ 7 days       │ Extended by editor/authority     │
└──────────────────────┴──────────────┴──────────────────────────────────┘

Cosmos DB TTL:
  Set TTL on event documents → auto-deleted by Cosmos DB
  On confirmation: update expires_at, Cosmos DB adjusts TTL
```

---

5) Scaling & Performance

Traffic Hotspot Handling

```
Problem: Rush hour in NYC → 10× more probes per segment than rural Iowa.

  ┌──────────────────────────────────────────────────────────┐
  │ ADAPTIVE SCALING                                         │
  │                                                          │
  │ 1. Event Hubs: Auto-inflate throughput units             │
  │    Base: 100 TU → Peak: 500 TU (auto-scale)              │
  │                                                          │
  │ 2. AKS: KEDA-based autoscaling                           │
  │    Scale GPS Ingestion pods on Event Hub lag             │
  │    Scale Routing pods on request queue depth             │
  │    Scale Traffic Aggregator on probe backlog             │
  │                                                          │
  │ 3. Redis: Cluster with read replicas per region          │
  │    Write to primary → async replicate to read replicas   │
  │    Route queries read from nearest replica               │
  │                                                          │
  │ 4. City-level isolation:                                 │
  │    Each major metro can have its own AKS namespace       │
  │    or dedicated node pool — prevents noisy neighbors     │
  └──────────────────────────────────────────────────────────┘
```

Data Freshness Pipeline

```
End-to-end latency for traffic updates:

  Driver GPS probe generated                    T = 0 ms
    │
    ├─ Batched on device (3-5 probes)           T = 0-5000 ms
    │
    ├─ Sent to API Gateway                      T = 5000 ms
    │
    ├─ Published to Event Hubs                  T = 5050 ms
    │
    ├─ Consumed by Traffic Aggregator           T = 5100 ms
    │
    ├─ Aggregated (30-sec tumbling window)      T = 5100-35000 ms
    │
    ├─ Written to Redis                         T = 35050 ms
    │
    ├─ Read by next routing query               T = 35050+ ms
    │
    └─ Reflected in traffic map tile            T = next tile refresh
                                                  (15-30 sec)

  Worst case probe-to-map: ~60 seconds
  Typical probe-to-routing: ~30 seconds
  User events: ~5-10 seconds (bypasses aggregation)
```

---

6) Privacy & Compliance

```
GPS data is extremely sensitive — a probe trail IS a movement diary.

┌──────────────────────────────────────────────────────────────────────┐
│                     PRIVACY ARCHITECTURE                             │
│                                                                      │
│  1. ANONYMIZATION                                                    │
│     • Probes stripped of user_id after map-matching                  │
│     • Traffic aggregation uses only (segment_id, speed)              │
│     • Individual traces NOT stored long-term                         │
│     • Random probe sampling (send 1/3 probes in low-density areas)   │
│                                                                      │
│  2. DATA MINIMIZATION                                                │
│     • Raw GPS probes: deleted after aggregation (< 24 hours)         │
│     • Aggregated traffic: keep 2 years (no individual identity)      │
│     • User reports: pseudonymized (reporter_id is opaque token)      │
│     • Route history: stored encrypted, user-deletable                │
│                                                                      │
│  3. GDPR / REGIONAL COMPLIANCE                                       │
│     • Data residency: EU probes processed in EU regions only         │
│     • Right to erasure: user deletion removes all associated data    │
│     • Consent: explicit opt-in for GPS collection                    │
│     • Data portability: export route history as JSON/CSV             │
│                                                                      │
│  4. AZURE SERVICES                                                   │
│     • Azure Key Vault: encryption keys for GPS data at rest          │
│     • Azure Entra ID: RBAC for internal access to probe data         │
│     • Azure Policy: enforce data residency per region                │
│     • Private endpoints: probe data never traverses public internet  │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

---

7) Interview Discussion Points

Why is Waze Architecturally Interesting?

```
1. WRITE-HEAVY at extreme scale
   7M GPS writes/sec — far more writes than reads
   Most systems are read-heavy; Waze inverts this.

2. REAL-TIME data pipeline with hard latency SLAs
   Stale traffic data = wrong routes = user frustration
   30-second freshness target is aggressive at scale.

3. CROWD-SOURCED data quality
   User reports need verification without ground truth.
   Confidence scoring + GPS corroboration is elegant.
```

```
4. GRAPH ALGORITHMS at production scale
   Contraction Hierarchies is a real CS contribution.
   Customizable CH for live traffic is non-trivial.

5. FEEDBACK LOOP
   More users → more probes → better traffic data → better routes
   → more users (network effect / flywheel)

6. PRIVACY vs. UTILITY tension
   Need individual GPS for traffic, but must protect privacy.
   Aggregation + anonymization + TTL is the standard approach.
```

Common Follow-up Questions

```
Q: How would you handle a country-wide natural disaster?
A: • Event system would auto-detect mass slowdowns
   • Manual "mega-event" override by operations team
   • Route avoidance zones for affected areas
   • Offline mode critical — connectivity likely disrupted
   • Pre-cached map data + last-known-good routes

Q: How does Waze differ from Google Maps architecturally?
A: • Google Maps has Street View, transit, indoor maps (broader scope)
   • Google uses its own fleet (Street View cars) for road data
   • Waze relies more heavily on community for map edits + events
   • Google has satellite imagery for traffic; Waze is probe-only
   • Post-acquisition: Waze probe data feeds INTO Google Maps

Q: How would you handle GPS spoofing / fake reports?
A: • Cross-validate GPS speed with accelerometer/gyroscope
   • Flag probes with impossible speeds (> 300 km/h) or teleportation
   • Reporter reputation system (low-trust users need more confirmations)
   • ML anomaly detection on probe patterns
   • Rate-limit reports per user (max 10/hour)

Q: What if Redis goes down?
A: • Routing falls back to historical traffic model (Cosmos DB)
   • Traffic map shows "historical" overlay with disclaimer
   • GPS probes buffer in Event Hubs (retention: 24 hours)
   • Redis cluster with replicas — single-node failure is transparent
   • Recovery: replay from Event Hubs to rebuild Redis state

Q: How would you add EV charging station routing?
A: • Add "battery_level" to route constraints
   • Graph edges gain "elevation_change" attribute (affects range)
   • Routing engine: if remaining_range < distance_to_dest,
     insert charging stop (treated like a waypoint)
   • Real-time charging station availability via partner APIs
   • Trade-off: fastest route vs. most energy-efficient route
```

---

8) Cost Optimization

```
┌──────────────────────────────────────────────────────────────────┐
│                    COST OPTIMIZATION STRATEGIES                  │
│                                                                  │
│  1. PROBE SAMPLING                                               │
│     Don't need every probe from every driver.                    │
│     • On well-covered roads (>10 probes/min): sample 1 in 3      │
│     • On sparse roads: keep all probes                           │
│     • Savings: ~40% reduction in Event Hubs throughput           │
│                                                                  │
│  2. TIERED STORAGE                                               │
│     • Hot (0-24h): Event Hubs + Redis → $$$$                     │
│     • Warm (1-30d): Cosmos DB (auto-tier) → $$                   │
│     • Cold (30d-2y): Data Lake Gen2 (Parquet) → $                │
│     • Archive (>2y): Blob Storage cool/archive tier → ¢          │
│                                                                  │
│  3. TILE CACHING                                                 │
│     • CDN hit rate target: 95%+ for map tiles                    │
│     • Traffic tiles: shorter TTL but still cacheable (15-30s)    │
│     • Pre-render popular zoom levels; generate rare tiles on demand │
│                                                                  │
│  4. COMPUTE RIGHT-SIZING                                         │
│     • GPS ingestion: CPU-light, I/O-heavy → small VMs, many pods │
│     • Routing engine: CPU-heavy, memory-heavy → large VMs, fewer │
│     • Traffic aggregation: memory-heavy → memory-optimized VMs   │
│     • Scale to zero overnight (non-peak regions)                 │
│                                                                  │
│  5. REGIONAL ISOLATION                                           │
│     • Deploy per-region, not globally replicated                 │
│     • Traffic data doesn't need cross-region sync                │
│     • Route computation is always local                          │
│     • Only user accounts need global replication                 │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

9) Monitoring & Observability

Key Metrics to Monitor:

Metric	Alert Threshold
Probe ingestion rate	Drop > 20% from expected → alert
Probe-to-Redis latency	P99 > 60 sec → alert
Route computation P99	> 500ms → alert
Reroute trigger rate	Spike > 5× normal → investigate
Event Hub consumer lag	> 10 sec behind → scale consumers
Redis memory usage	> 80% → alert

```
| CDN cache hit rate       | < 90% for tiles → investigate       |
| ETA accuracy (actual vs  | Error > 5 min for >10% trips → fix  |
|   predicted)             |                                     |
| False event report rate  | > 30% rejected → tune confidence    |
└──────────────────────────┴─────────────────────────────────────┘
```

Azure Services:

- Azure Monitor + Application Insights (metrics, traces, logs)
- Azure Data Explorer (real-time dashboards on probe quality)
- Azure Alerts → PagerDuty/Teams for on-call
- Grafana dashboards for traffic coverage heatmaps